1. El primer protocolo de aplicación que vemos es DNS y el protocolo de transporte utilizado es UDP.

2. **Relación dentro de las cabeceras:** Se observa una inversión simétrica entre origen y destino para que la respuesta pueda regresar. El puerto origen del cliente pasará a ser el puerto destino del servidor, y la IP origen (`192.168.0.1`) será la IP destino de la respuesta.
   No se realiza una conexión ya que UDP es un protocolo no orientado a conexión (*connectionless*) y no establece handshake previo.
   Socket PC: (192.168.0.1 : 1025) ⇔ Socket Servidor DNS: (192.168.0.3 : 53)

3. representa la carga útil de la capa 7. Contiene la consulta DNS Query solicitando la dirección IP asociada a `[www.mipc.com](https://www.mipc.com)`

4. El servidor DNS responde con un DNS Answer que contiene la IP asignada (`192.168.0.2`). En la cabecera de transporte UDP se invierten los puertos: Src Port pasa a ser 53 y Dst Port pasa a ser 1025.

5. El segundo protocolo de aplicación que vemos es HTTP y el protocolo de transporte utilizado es TCP.

6. **Relación en cabeceras:** Se observa la inversión simétrica de direcciones IP y puertos, además de la sincronización estricta entre números de secuencia (SEQ) y números de acuse de recibo (ACK).
   Si se realiza una conexión ya que TCP es un protocolo orientado a conexión y requiere establecer sesión previa mediante el saludo de tres vías (*Three-Way Handshake*) antes de enviar los datos de la página web.
   Socket PC: (192.168.0.1 : Puerto_TCP_Cliente) ⇔ Socket Servidor Web: (192.168.0.2 : 80)

7. Sí, ahora sí se observa. Aparecen los 3 segmentos puros de control de la capa de transporte:
   PC → **Servidor Web:** Flag SYN.

   Servidor Web → **PC:** Flags SYN, ACK.

   PC → **Servidor Web:** Flag

8. No, no aparecen nuevos mensajes de ambos. No vemos mensajes adicionales del protocolo UDP; únicamente aparecen mensajes nuevos de TCP.
   ¿Por qué?**:** UDP es un protocolo sin conexión (*connectionless*) y no genera paquetes de control de establecimiento ni de cierre de sesión. TCP, por el contrario, requiere segmentos de control para iniciar (SYN) y finalizar (FIN) la conexión.
   Socket PC: (192.168.0.1 : Puerto_TCP_Cliente) ⇔ Socket Servidor Web: (192.168.0.2 : 80)

9. **ISN (*Initial Sequence Number*):**
   SEQ = 0 para el SYN inicial de la PC y SEQ = 0 para el SYN, ACK del servidor.
   Flags activos:
   - **PC → SRV:** SYN.
   - **SRV → PC:** SYN, ACK.
   - **PC → SRV (tercer paso):** ACK.

   Conexión establecida: La conexión queda en estado ESTABLISHED a partir de que la PC envía el segmento con SEQ = 1 (ACK de confirmación).

10. Inicio del cierre: Se inicia cuando se transmite un segmento TCP con la bandera FIN, ACK activada (generalmente enviado por el servidor web tras terminar de transmitir la página HTTP o por la PC).

    ¿Cuándo se da por cerrada?: Se da por cerrada cuando el host receptor responde con su propio segmento FIN, ACK y se confirma con el ACK final, cambiando los sockets al estado CLOSED.

11. En la PC **(**192.168.0.1**):** TCP 192.168.0.1:Puerto_Cliente 192.168.0.2:80 ESTABLISHED**.**
En el Servidor **(**192.168.0.2**):** TCP 192.168.0.2:80 192.168.0.1:Puerto_Cliente ESTABLISHED**.**

12. Al conectar una segunda PC (192.168.0.4) y solicitar la página web a la vez, se establecen 2 conexiones TCP independientes en el servidor web.

13.
$Conexión\ 1: (192.168.0.1: Puerto\ PC1, 192.168.0.2: 80)$

$Conexión\ 2: (192.168.0.4: Puerto\ PC2, 192.168.0.2: 80)$